

BACKEND DEVELOPER PORTFOLIO

최시원

Backend Developer

AI 경험 제외

JAVA SPRING MYSQL POSTGRESQL REDIS

PROJECT SUMMARY

01 / AUTHORIZATION

HDP ERP

User Service 원본과 Redis UserSnapshot으로 권한을 조회했습니다. 정보 변경은 캐시 무효화로 반영하고 삭제 실패는 Outbox로 재처리했습니다.

02 / LOCATION SEARCH

ToiletNearBy

조회 범위와 캐시 대상을 조정하고, k6 부하 시험으로 위치 검색 응답 시간을 비교했습니다.

03 / MATCH QUERY

GAME_MATCH

실제 목록 API의 반복 SQL을 계측하고 CTE 조회안과 응답을 자동 대조했습니다. 동시 승인 전략은 별도 실험으로 비교했습니다.

프로젝트별 읽는 순서

프로젝트 소개에서 서비스와 역할을 확인한 뒤 대표 해결 사례로 이어집니다. ERP는 담당한 플랫폼 기능이 많아 구현 범위를 한 페이지 더 정리했습니다.

01 **HDP ERP** P. 3–5

담당 범위와 대표 해결 사례

ERP 소개 → 계정과 인증 구성 → 권한 조회와 실패 복구

02 **ToiletNearBy** P. 6–7

위치 검색 성능 개선

서비스 소개 → 후보 재사용과 검색 응답 개선

03 **GAME_MATCH** P. 8–10

목록 조회와 동시 승인 검토

서비스 소개 → N+1, EntityGraph와 CTE → 비관적 락 선택

자동차 부품 주문과 재고 관리

가상 자동차 부품 회사 현대파츠의 지점과 본사가 사용하는 ERP입니다. 고객 주문부터 재고 이동, 구매 발주와 생산 작업까지 5개 서비스로 나눴습니다.

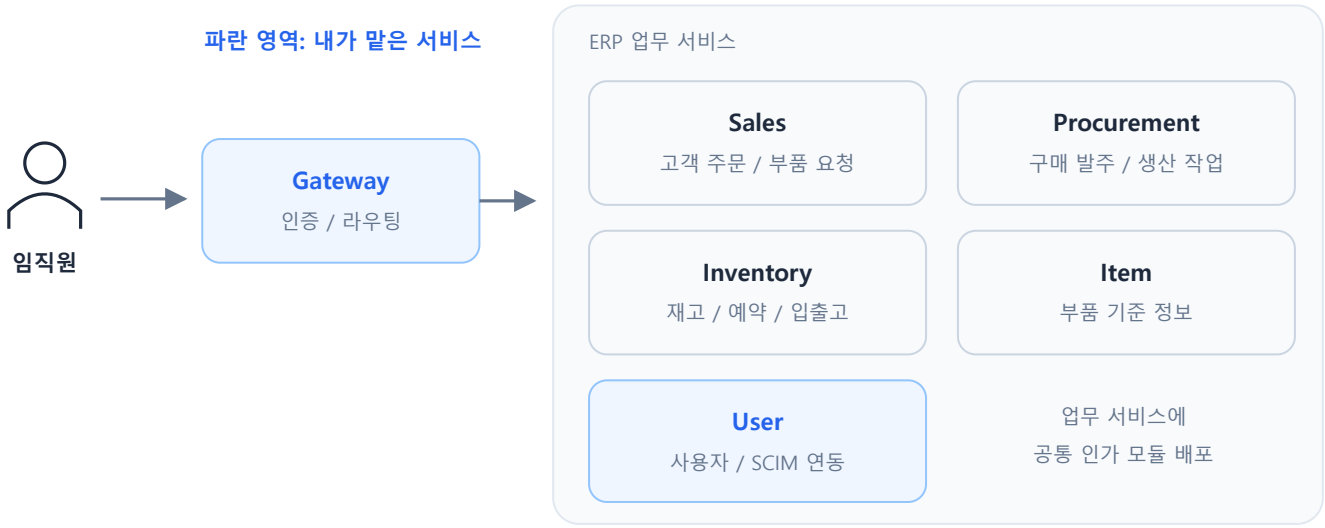
담당 5인 팀에서 User Service, Gateway와 경량 프레임워크를 맡았습니다. 로그인 연동, 사용자 정보 동기화, 업무 API의 권한 검증을 구현했습니다.

서비스 이용 흐름

고객 주문 → 지점 재고 확인 → 본사에 부품 요청 → 부품 확보와 이동

지점에 재고가 있으면 바로 출고합니다. 본사 재고도 부족하면 구매 또는 생산 작업을 진행합니다.

서비스 구성과 담당 영역



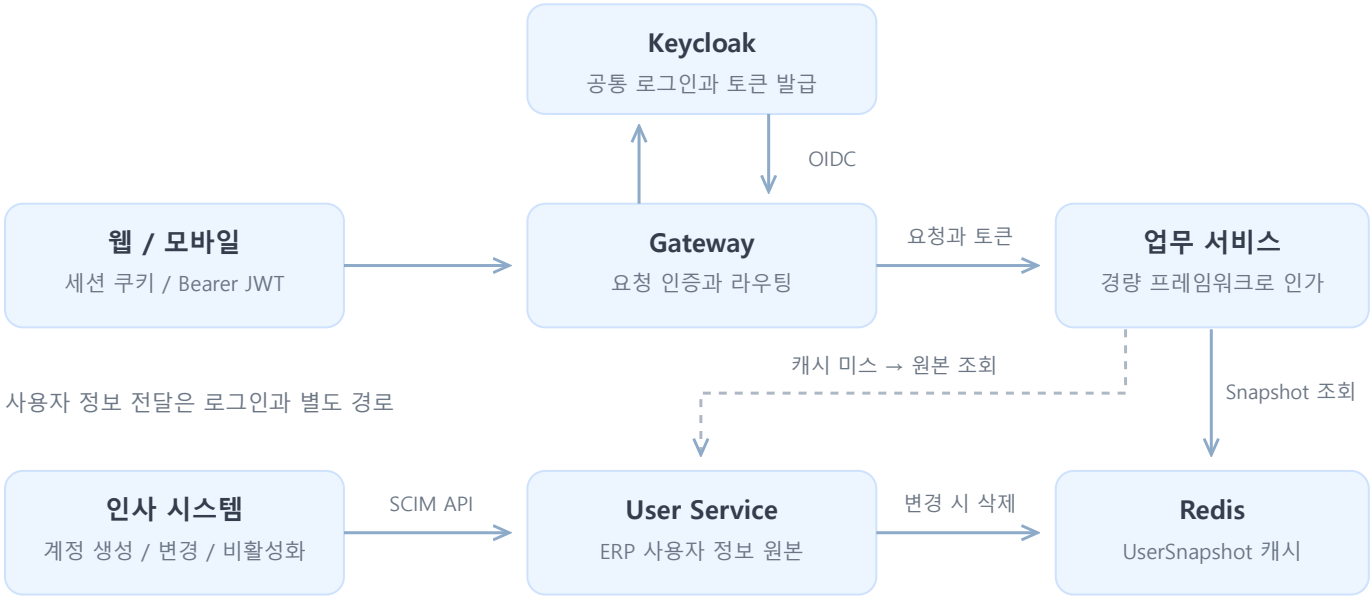
구현 범위 이 가운데 계정 연동과 인증, 업무 권한을 맡았습니다. 다음 장부터 공통 로그인과 ERP 사용자 정보를 나눈 구조, 권한 조회와 변경 반영 과정을 설명합니다.

계정 연동과 인증 구조

ERP 사용자 관리, Gateway와 경량 프레임워크를 개발했습니다. 회사 공통 로그인과 ERP 업무용 사용자 정보를 분리하고 각 서비스의 권한 조회를 연결했습니다.

구조 Keycloak은 로그인과 토큰 발급을, User Service는 ERP 사용자 정보를 관리합니다. Gateway는 요청 인증과 라우팅을 맡고, 업무 권한은 각 서비스의 공통 모듈에서 판단합니다.

ERP의 인증과 사용자 정보 흐름



설계의 중심

로그인은 Keycloak에, ERP 사용자 정보는 User Service에 두었습니다. 각 업무 서비스는 공통 모듈을 통해 같은 방식으로 상태와 권한을 확인합니다.

문제 상황

토큰 권한 반영 지연

→

해결 방법

UserSnapshot 조회

인사 정보에 따라 ERP 업무 권한이 달라지는 환경에서, 권한 정보의 최신성과 반복 조회 비용을 함께 다뤘습니다.

문제

JWT의 권한은 만료 전까지 바뀌지 않습니다. 처음에는 매 요청 User Service를 조회했지만 호출 비용이 반복돼, User DB를 원본으로 유지하고 Redis에 UserSnapshot을 두었습니다.

평소의 인가 요청

업무 API
@RequireRole

→

Redis Snapshot
적중하면 이 정보로 인가

미스 / 장애 →

User Service
User DB의 원본 조회

인사 정보가 바뀌면

SCIM / User DB
변경과 삭제 작업 저장

→

Snapshot 삭제
실패하면 Outbox 재시도

→

다음 조회
변경된 원본으로 재생성

구현

같은 인가 경로를 공통 모듈로 제공

서비스는 허용 역할을 선언하고, 모듈은 Snapshot 조회와 상태 검증을 담당합니다. Redis와 User Service 모두 확인할 수 없으면 요청을 허용하지 않습니다.

결과

권한 Snapshot 조회 p95

60.89 → 32.81ms

캐시 적중 시 원본 호출

0회

직접 조회 경로는 2932회

반복 원본 조회는 줄이고, 사용자 변경은 캐시 삭제와 Outbox 재시도로 반영하도록 구성했습니다.

조건과 범위

동일 사용자 1명, 30초, 검증용 HTTP 진입점 기준입니다. ERP 전체 API 성능은 아닙니다. 삭제 뒤 구버전 Snapshot이 재저장되는 경쟁은 재현됐으며 상세본에 미해결 과제로 구분했습니다.

주변 화장실 검색과 후기 공유

현재 위치나 검색한 장소 주변의 공중화장실을 찾는 웹 서비스입니다. 상세 정보와 후기를 확인하고 별점, 비밀번호 등 이용 정보를 공유할 수 있습니다.

3인 팀 백엔드 개발 / 주변 검색, 장소 검색 연동과 JWT 인증 / 이후 개인 리팩터링

주변 검색의 처리 범위



초록 점은 결과에 포함, 모서리의 빨간 점은 제외
계정 인증이 필요한 후기와 별점은 별도 API로 처리

주요 기능	서비스 설명
주변 화장실 검색	현재 위치와 반경을 기준으로 공중화장실을 찾고 상세 정보 확인
장소 검색	Kakao Local API로 장소를 검색하고 해당 위치 주변의 화장실 탐색
후기와 이용 정보	후기, 별점과 비밀번호 등 이용 정보를 작성하고 공유
사용자 인증	JWT 기반 로그인과 사용자 식별, 작성 기능의 접근 제어

개선 대상 이 가운데 주변 화장실 검색의 반복 조회 비용을 줄였습니다. DB 조회 범위를 축소한 뒤 격자별 후보 캐시를 적용하고, 실제 위치와 반경은 요청마다 다시 계산했습니다.

문제 상황

인접 요청의 반복 DB 조회

→

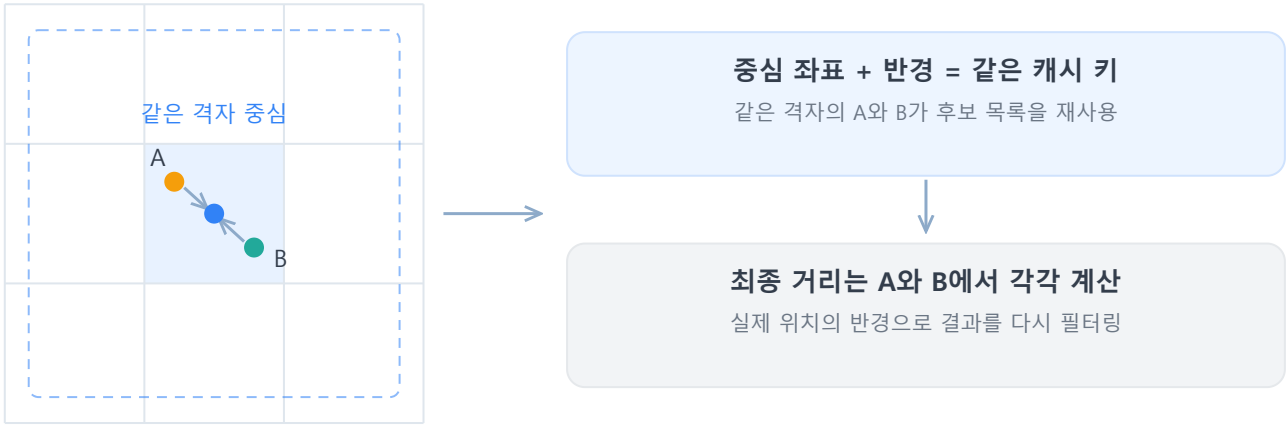
해결 방법

격자 중심 후보 캐시

실제 좌표를 격자 중심으로 정규화해 인접 요청이 같은 후보 목록을 재사용하도록 했습니다.

문제 좌표를 그대로 키로 쓰면 가까운 요청도 캐시가 나뉩니다. 0.005도 격자의 중심 좌표와 반경을 키로 사용하고, 최종 거리는 실제 사용자 위치에서 다시 계산했습니다.

격자 중심 정규화와 요청별 거리 계산



파선: 중심 이동을 보완한 후보 조회 범위

격자 중심은 후보 조회에만 사용합니다. 도식은 비례를 단순화했으며 최종 검색 위치는 A와 B 그대로입니다.

결과

검색 응답 p95

41.57 → 8.52ms

68개 좌표의 결과 불일치

0건

후보 목록은 재사용하고 최종 결과는 요청별로 계산했습니다. 경계와 모서리에서도 전체 탐색과 같은 대상, 같은 반올림 거리를 반환했습니다.

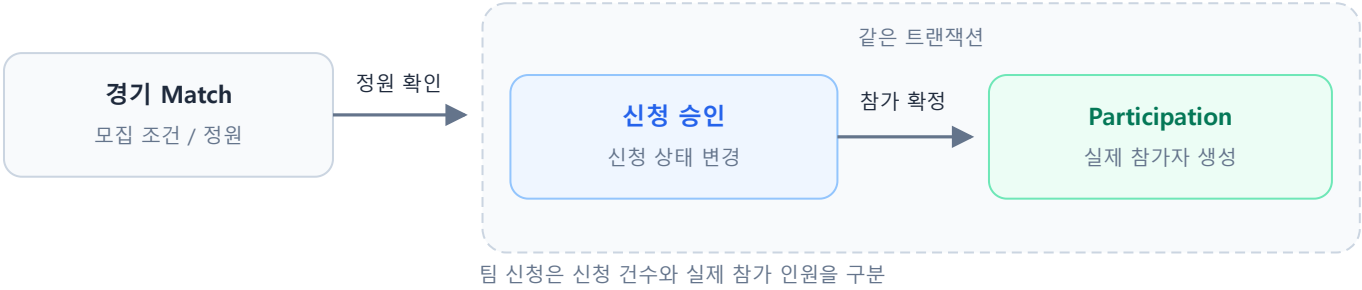
조건과 범위 2026-10-31 로컬 PostgreSQL 17.5, 합성 10만 건, k6. 고정 좌표, 반경 3km, warm cache의 단일 부하 시험입니다. 68좌표 정확성 검증은 별도 실행입니다.

온라인 게임 친선경기 매칭

게임 이용자가 친선경기를 만들고 참가를 신청하는 커뮤니티 서비스입니다. 게임별 프로필과 경기 모집, 게시판, 댓글, 친구 기능을 제공합니다.

1인 백엔드 개발 / 게임 프로필, 친선경기와 커뮤니티 API / 이후 조회와 동시성 리팩터링

신청에서 참가 확정까지



팀 신청은 신청 건수와 실제 참가 인원을 구분

주요 기능	서비스 설명
게임 프로필	게임별 사용자 프로필을 등록하고 친선경기 활동에 사용
친선경기 모집	게임과 일정, 모집 조건을 등록하고 경기 목록과 상세 정보 조회
신청과 참가 관리	참가 신청과 승인, 거절, 취소를 처리하고 승인된 참가자 관리
커뮤니티	게시글과 댓글 작성, 친구 관계 등 게임 이용자 간 소통 기능

개선 대상 목록 조회에서는 N+1을 EntityGraph로 줄인 뒤 남은 집계를 CTE로 구성했습니다. 참가 승인에서는 정원 초과를 막는 두 락 방식의 비용을 비교하고 짧은 승인 트랜잭션과 재시도 비용을 고려해 비관적 락을 선택했습니다.

이어서 조회 개선

03

GameMatch

온라인 게임 친선경기 매칭

문제 상황

N+1과 반복 집계

→

해결 방법

EntityGraph + CTE

목록의 주최자와 게임 정보, 신청 현황을 조립하면서 발생하는 반복 조회를 단계적으로 줄였습니다.

문제

목록을 순회하며 연관 정보를 읽는 N+1과 경기별 집계 호출이 함께 있었습니다. EntityGraph는 연관 조회를 합쳤지만 신청 수와 최근 변경 시각의 반복 집계는 남았습니다.

조회 개선 순서

N+1 발견	→	EntityGraph	→	CTE 집계
주최자와 게임 추가 조회		연관 정보 함께 조회		페이지 경기만 일괄 집계

구현

연관 조회와 집계를 나눠 개선

주최자와 게임은 EntityGraph로 가져오고, 유효 신청 수와 상태별 수, 최근 변경 시각은 페이지 대상 경기를 먼저 정한 뒤 CTE로 묶었습니다.

결과

별도 SQL 재현 실험

121 → 81 → 1회

기존 API와 CTE 결과 대조

20건 일치

연관 조회를 합치는 것만으로 남는 집계 호출까지 일괄 조회안으로 구성했습니다. ID, 순서와 집계 조건을 맞춰 응답의 의미를 대조했습니다.

조건과 범위

121 → 81 → 1은 기존 호출 구조의 SQL 재현 실험입니다. 실제 EntityGraph 레거시 API는 상태 확인을 포함해 안정 상태 101회였습니다. CTE 적용 후 Java API 전체 성능은 아직 측정하지 않았습니다.

이어서 비관적 락 선택

문제 상황

동시 승인 시 정원 초과

→ 해결 방법

비관적 락 선택

짧은 승인 트랜잭션에서 같은 경기의 변경을 순차 처리하도록 비관적 락을 선택했습니다.

문제

두 관리자가 같은 경기의 남은 정원을 동시에 읽으면 초과 승인할 수 있습니다. 짧은 승인 작업에서 낙관적 락은 정원이 남아도 버전 충돌로 롤백과 재실행이 생길 수 있어, 최신 상태를 기다려 확인하는 비관적 락을 선택했습니다.

구현

시간보다 처리 흐름을 우선한 선택

외부 호출 없이 정원 확인과 승인 상태, 참가 정보를 같은 트랜잭션에서 처리하는 조건입니다. 잠시 기다렸다가 최신 상태를 읽는 방식으로 충돌 후 재실행과 오류 처리 부담을 줄이고자 했습니다.

결과

낙관적 락 HTTP 평균

5.55ms

비관적 락 HTTP 평균

4.65ms

별도 HTTP 시험에서 두 방식 모두 정원 초과와 중복 참가가 없었습니다. 마지막 한 자리 경쟁에서는 비관적 락이 재실행 없이 한 건 승인과 한 건 정원 마감을 처리했습니다.

두 방식의 선택 기준

전략	얻는 점	감수할 비용
비관적 락 선택	대기 후 최신 정원으로 판단	행 잠금 획득과 경합 시 대기
낙관적 락	사전 조회 잠금 없이 충돌 감지	충돌 요청의 롤백과 재시도

조건과 범위

로컬 MySQL 8.0.33, 테스트 전용 Spring HTTP 승인 경로입니다. 전략별 940요청의 전체 평균이며 인위적 지연은 없습니다. 실제 제품 승인 API의 구현 완료나 일반적인 성능 우위를 의미하지 않습니다.